

Transcripción de operadores

Clara Rodríguez, Alejandro Hernández y Lucía Martínez

15 de octubre de 2025

Resumen

Este documento está destinado para la transcripción de operadores C a su equivalente en lenguaje circom.

Guía de contenidos

1. Notación
2. Operadores Lógicos
3. Operadores de Comparación
4. Operadores Aritméticos
5. Acceso a Arrays
6. Expresiones condicionales
7. Operadores de desplazamiento

1. Notación

A lo largo del documento estudiaremos cómo traducir instrucciones y expresiones generadas por un lenguaje imperativo simple en un conjunto de ecuaciones en formato R1CS equivalente definimos las siguientes funciones:

Definición 1 Definimos la función de traducción de expresiones $TC : Expr \rightarrow C \times Expr$ siendo:

- C un sistema de ecuación en formato R1CS
- C e una expresión lineal

La intuición es que en caso de que las constraints C se verifiquen entonces e y e' siempre tienen el mismo valor.

Vamos a definir la función TC de forma inductiva, comenzando por los casos base (números y variables) para luego pasar a definir los casos compuestos.

Casos base

- Si la expresión es un número n :

$$TC(n) = (TC(\emptyset, n))$$

- Si la expresión es una variable v :

$$TC(v) = (TC(\emptyset, v))$$

2. Operadores Lógicos

Operación	Deducción
$e = \text{not } b$	$\frac{TC(e_1) = (C, e'_1)}{TC(\text{not } e_1) = (C, 1 - e'_1)}$
$e = \text{and } b$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \text{and } e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} = e'_1 * e'_2\}, v_{aux})}$
$e = \text{or } b$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \text{or } e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} = e'_1 * e'_2\}, e'_1 + e'_2 - v_{aux})}$
$e = e_1 \& e_2$	$\frac{TC(e_1) = (C_1, e_{1-\text{binario}}) \quad TC(e_2) = (C_2, e_{2-\text{binario}})}{TC(e_1 \& e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} = e_{1-\text{binario}}[i] * e_{2-\text{binario}}[i]\}, v_{aux})}$
$e = e_1 \mid e_2$	$\frac{TC(e_1) = (C_1, e_{1-\text{binario}}) \quad TC(e_2) = (C_2, e_{2-\text{binario}})}{TC(e_1 \mid e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} = e_{1-\text{binario}}[i] * e_{2-\text{binario}}[i]\}, e_{1-\text{binario}}[i] + e_{2-\text{binario}}[i] - v_{aux})}$

3. Operadores de Comparación

Un detalle a observar en las funciones *LessThan()* y *LessThanEq()* es que para números negativos podría presentar problemas.

El funcionamiento de *Num2Bits(n)* cuando recibe un número negativo es el siguiente:

- El número negativo se interpreta como un cuerpo finito.
- Éste se interpretará como un número superior a $2^n - 1$, saliéndose fuera del rango $[0, 2^n - 1]$, por tanto la constraints no se pueden cumplir y la prueba no se generará.
- En conclusión, con *Num2Bits()* garantizamos el uso de número positivos dentro del rango mencionado de manera implícita.

Observación: Gracias a la constraint $lc1 === in$ (fuerza a que la entrada cumpla el rango $[0, 2^n - 1]$), podemos evitar el uso de número negativos, ya que estos no cumplirán tal restricción.

Num2Bits(n):

```

1      template Num2Bits(n){
2          signal input in;
3          signal output out[n];
4          var lc1=0; // Reconstruye el numero decimal a convertir para la
                    constraint
5
6          var e2=1; // Evitar la potencia
7          for (var i = 0; i<n; i++) {
8              out[i] <-- (in >> i) & 1; // Contruimos el numero en binario
9              out[i] * (out[i] -1 ) == 0;
10             lc1 += out[i] * e2;
11             e2 = e2+e2;
12         }
13
14         lc1 === in; // Constraint mencionada anteriormente
15     }
```

Partiendo de esta información, podemos realizar pequeñas modificaciones en las funciones *LessThan()* y *LessThanEq()* para asegurar el uso de enteros positivos.

1	<code>template SecureLessThan(n){</code>	1	<code>template SecureLessThanEq(n)</code>
2	<code>signal input in[2];</code>	2	<code>{</code>
3	<code>signal output out;</code>	3	<code>signal input in[2];</code>
4		4	<code>signal output out;</code>
5	<code>// -----</code>	5	<code>// -----</code>
6		6	<code>component checkA =</code>
7	<code>Num2Bits(n);</code>	7	<code>Num2Bits(n);</code>
8	<code>checkA.in <= in[0];</code>	8	<code>checkA.in <= in[0];</code>
9		9	
10	<code>component checkB =</code>	10	<code>component checkB =</code>
11	<code>Num2Bits(n);</code>	11	<code>Num2Bits(n);</code>
12	<code>checkB.in <= in[1];</code>	12	<code>checkB.in <= in[1];</code>
13		13	
14	<code>adder = BinSum(n, 2);</code>	14	<code>adder = BinSum(n, 2);</code>
15		15	
16	<code>var i;</code>	16	<code>var i;</code>
17	<code>for (i=0;i<n;i++) {</code>	17	<code>for (i=0;i<n;i++) {</code>
18	<code>checkA.out[i] ==> adder</code>	18	<code>checkA.out[i] ==> adder.</code>
19	<code>in[0][i];</code>	19	<code>in[0][i];</code>
20	<code>checkB.out[i] ==> adder</code>	20	<code>checkB.out[i] ==> adder.</code>
21	<code>in[1][i];</code>	21	<code>in[1][i];</code>
22	<code>}</code>	22	<code>}</code>
23		23	
		24	
		25	
		26	
			<code>// Miramos el bit +</code>
			<code>significativo (MSB), 1</code>
			<code>si A < B, 0 de lo</code>
			<code>contrario</code>
	<code>adder.out[n-1] ==> out;</code>		<code>component equal = IsEqual</code>
	<code>}</code>		<code>();</code>
			<code>equal.in[0] <= in[0];</code>
			<code>equal.in[1] <= in[1];</code>
			<code>out <= adder.out[n-1] +</code>
			<code>equal.out - adder.out[</code>
			<code>n-1] * equal.out;</code>
			<code>}</code>

Con esta pequeña modificación, al usar para ambos número a comparar la función *Num2Bits()* nos aseguramos de que se cumpla la restricción de ser número enteros positivos dentro del rango.

Operación	Deducción
$e = e_1 = e_2$	Para un component zero = IsZero($e_1 - e_2$): $\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 = e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} \leq zero.out\}, v_{aux})}$
$e = e_1 \neq e_2$	Para un component zero = IsZero($e_1 - e_2$): $\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \neq e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} \leq zero.out\}, 1 - v_{aux})}$
$a < b$	Para un component lessThan = SecureLessThan(n)(a,b) : $\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 < e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} \leq lessThan.out\}, v_{aux})}$
$a \leq b$	Para un component lessThanEq = SecureLessThanEq(n)(a,b) : $\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \leq e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} \leq lessThanEq.out\}, v_{aux})}$
$a > b$	Para un component lessThanEq = SecureLessThanEq(n)(a,b) : $\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 > e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} \leq lessThanEq.out\}, 1 - v_{aux})}$
$a \geq b$	Para un component lessThan = SecureLessThan(n)(a,b) : $\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \geq e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} \leq lessThan.out\}, 1 - v_{aux})}$

Nota: Las funciones IsZero() son circuitos que comparan si un valor es igual a 0.

Nota: Las funciones IsEqual()) comprueba si dados dos números, estos son iguales o no.

4. Operadores Aritméticos

Tanto en C como en Circom, los operadores suma (+), resta (-) y multiplicación (*) no difieren. En cambio, en operaciones como la división entera (/) y el módulo (%) si que encontramos diferencias y por ende es necesaria su transcripción.

La idea desde la que se parte para encontrar esta equivalencia es de la propiedad fundamental de la división entera, la cual establece:

$$a = b \cdot q + r$$

Operación división

Partiendo de la fórmula $a = b * q + r$, podemos llegar al resultado de la operación a/b forzando a que se cumpla la fórmula pero despreciando el resto (ya que se trata de división entera).

El término "forzar" hace referencia a usar una variable en Circom, de forma que partiendo del dividendo y del divisor, el cociente solo pueda tener ese valor.

Ejemplo de código en Circom:

```

1      template IntegerDivision() {
2          signal input a; // Dividendo
3          signal input b; // Divisor
4          signal output c; // Cociente q
5
6          var q;
```

```

7         a === b * q;
8         c <= q;
9     }
10
11     component main = IntegerDivision();

```

Operación módulo

Partiendo de la fórmula $a = b \cdot q + r$, podemos llegar al resultado de la operación $a \% b$ con la variable r , la que representa el resto de una división.

Por otro lado, ha de cumplirse la expresión $0 \leq r < b$. De lo contrario, podría producirse problemas en cuanto a la seguridad del circuito.

$$a = b \cdot q + r \quad (1)$$

$$r = a - (b \cdot q) \quad (2)$$

$$r = a - (b \cdot (a/b)) \quad (3)$$

Nota: q representa el resultado de una división, por lo que podemos decir que $q = a/b$.

Para que se cumpla la restricción $0 \leq r < b$:

```

1     component lessThan = SecureLessThan(n);
2     lessThan.in[0] <= r;
3     lessThan.in[1] <= b;
4
5     // 1 si r < b, como ya se checkea en SecureLessThan que sea positivo o 0
6     // es suficiente
7
7     lessThan.out == 1 // Hacemos que se cumpla la restriccion,
8     // Si no se cumple, no se generara la prueba

```

Operación	Deducción
$e = a/b$	$\frac{TC(a) = (C_1, a') \quad TC(b) = (C_2, b') \quad TC(q) = (C_3, q') \quad TC(r) = (C_4, r')}{TC(a/b) = (C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge \{v_{aux} = b' * q'\}, a' = v_{aux} + r')}$
$e = a \% b$	$\frac{TC(a) = (C_1, a') \quad TC(b) = (C_2, b')}{TC(a \% b) = (C_1 \wedge C_2 \wedge \{v_{aux} = a' / b'\} \wedge \{v_{aux_2} = b * v_{aux}\}, a' - v_{aux_2})}$

5. Acceso a Arrays

La diferencia principal entre C y Circom en el acceso a arrays radica en que el tamaño de los arrays en Circom ha de ser fijo a la hora de compilar y ser siempre constante, mientras que en C puede ser estático y dinámico. Además, en Circom el valor de los arrays no puede ser reasignado dinámicamente.

Nota: En C, el tamaño de un array puede ser variable, ya sea por arrays de longitud variable o por el uso de memoria dinámica

1	<code>// Codigo en C</code>	1	<code>//Codigo circom</code>
2	<code>#include <stdio.h></code>	2	<code>template accesoArray(N) {</code>
3		3	<code>signal input lista[N]; //</code>
4	<code>int main() {</code>	4	<code>Tamanyo fijo, suponemos</code>
5			<code>iniciada</code>
6	<code>// Ejemplo de array con</code>	5	<code>signal output sumaAcumulada;</code>
	<code>tamanyo variable con</code>	6	
	<code>funcion f()</code>	7	<code>var suma = 0;</code>
7	<code>int n = f();</code>	8	<code>for(int i = 0; i < N; i++){</code>
8	<code>int lista[n]; // Se decide</code>	9	<code>suma = suma + lista[i];</code>
	<code>en tiempo de ejecucion</code>	10	
9		11	<code>// NO se puede hacer lo</code>
10			<code>siguiente</code>
11	<code>// EJemplo de array de</code>	12	<code>// lista[i] <== 3;</code>
	<code>tamanyo variable con</code>	13	<code>}</code>
	<code>asignacion de memoria</code>	14	
	<code>dinamica</code>	15	<code>sumaAcumulada <== suma;</code>
12	<code>int *lista = malloc(x);</code>	16	<code>}</code>
13		17	
14	<code>//x tamanyo que se quiera</code>	18	<code>component main = accesoArray</code>
	<code>reservar en memoria</code>		<code>(10);</code>
	<code>dinamica</code>		
15	<code>}</code>		

Nota: En Circom no se puede fijar el tamaño de un array con una señal.

En Circom, no se puede acceder directamente a posiciones de arrays con señales inputs. Sin embargo, se puede acceder indirectamente a esta posición sin usar esta señal input.

La propuesta para realizar este acceso es utilizar un multiplexor, el cuál tenga tantas entradas como tamaño tenga el array en cuestión. Por tanto, dada una posición el MUX devuelve el valor de la misma.

```

1      template MuxN(n) {
2          signal input in[n]; //
3          signal input sel;
4          signal output out;
5
6          signal isSel[n];
7          component eq[n];
8
9          for (var i = 0; i < n; i++) {
10             eq[i] = IsEqual();
11             eq[i].in[0] <== sel;
12             eq[i].in[1] <== i;
13             isSel[i] <== eq[i].out;
14         }
15
16         signal suma_parcial[n + 1];
17         suma_parcial[0] <== 0;
18
19         for (var i = 0; i < n; i++) {
20             suma_parcial[i + 1] <== suma_parcial[i] + isSel[i] * in[i];
21         }

```

```

22         out <= suma_parcial[n];
23     }
24
25
26     template acceso_lineal(n) {
27         signal input acceso;
28         signal input array[n];
29         signal output out;
30
31         // Asumimos acceso dentro del rango
32         assert(acceso >= 0);
33         assert(acceso < n);
34
35         component mux = MuxN(n);
36
37         for (var i = 0; i < n; i++) {
38             mux.in[i] <= array[i];
39         }
40         mux.sel <= acceso;
41
42         out <= mux.out;
43     }

```

6. Expresiones condicionales

En C, al ser un lenguaje de ejecución dinámica, los condicionales pueden decidir el valor final de una variable en tiempo de ejecución.

En Circom, cada signal representa un “cable” del circuito, que solo puede recibir un valor. Por ello, no es posible reasignar un signal dentro de un condicional dinámico.

Para simular decisiones condicionales basadas en entradas (inputs), se deben usar expresiones aritméticas que implementen la lógica de selección, de manera que el valor de salida quede correctamente definido sin reasignar el signal.

Un ejemplo específico de conversión de un condicional C a uno en Circom puede ser el siguiente:

<pre> 1 //Codigo en C 2 #include <stdio.h> 3 4 int main(int argc, char * 5 argv[]) { 6 int entrada = argv[1]; 7 char mensaje[50]; 8 9 if(entrada % 2 == 0){ 10 sprintf(mensaje, "%d es 11 par\n", entrada); 12 } 13 else { 14 sprintf(mensaje, "%d es 15 impar\n", entrada); 16 } 17 18 printf("s", mensaje); 19 free(mensaje); 20 21 return 0; 22 } </pre>	<pre> 1 //Codigo circom 2 template condicionales() { 3 signal input entrada; 4 signal output salida; //0 5 par, 1 impar 6 7 var var_division; 8 entrada == 2 * 9 var_division; 10 11 salida <= (entrada - (2* 12 var_division)) 13 } 14 15 component main = 16 condicionales(); </pre>
--	--

Con este ejemplo se ilustra como puede transformarse una expresión condicional que comprueba si un número es par o impar en una expresión aritmética que representa lo mismo y evita el reasignar la señal de salida más de una vez (algo no permitido).

A la hora de generalizar esta conversión condicional de C a Circom, primero tenemos que darnos cuenta del patrón que se sigue partiendo desde el caso más simple.

Caso más simple: if/else

```

1      //Codigo en C
2      if (age < 0) {
3          printf("Edad invalida.\n");
4      } else {
5          printf("Perfil: adulto mayor.\n");
6      }
7
8
9      //Codigo en Circom
10     component less_0 = LessThan()(age,0);
11
12     signal less_0 = less_0.out;
13     signal not_less_0 = 1 - less_0.out;
14
15     mensaje <= less_0 * m1 + not_less_0 * m2;

```

Como podemos ver, el patrón que sigue es crear una componente equivalente a la expresión lógica utilizada en el if. Con esta componente, al tener dos opciones, el if o el else, observamos que el else siempre será la suma de la expresiones anteriores menos 1, ya que al no cumplir las anteriores tiene que ser lo contrario.

Este caso es demasiado simple, por lo que un ejemplo con más condicionales detallará mejor lo explicado anteriormente.

La traducción formal de los condicionales es la siguiente:

Operación	Deducción
$e = \text{if } \textit{cond} \text{ then } e_1 \text{ else } e_2$	$\frac{TC(\textit{cond}) = (C_{\textit{cond}}, \textit{cond}') \quad TC(e_1) = (C_1, e_1') \quad TC(e_2) = (C_2, e_2')}{TC(\text{if } \textit{cond} \text{ then } e_1 \text{ else } e_2) = (C_{\textit{cond}} \wedge C_1 \wedge C_2 \wedge \{v_{aux} = \textit{cond}' * e_1' + (1 - \textit{cond}') * e_2'\}, v_{aux})}$

Caso complejo: if/else if/else

```

1      // Codigo en C
2
3
4      if (age < 0) {
5          printf("Edad invalida.\n");
6      } else if (age < 13) {
7          printf("Perfil: menor.\n");
8      } else if (age < 18) {
9          printf("Perfil: adolescente.\n");
10     } else if (age < 30) {
11         printf("Perfil: joven adulto.\n");
12     } else if (age < 60) {
13         printf("Perfil: adulto.\n");
14     } else {
15         printf("Perfil: adulto mayor.\n");
16     }

```

- Por cada condición de igualdad/desigualdad se crea un componente correspondiente:

```

1      component less_0 = LessThan()(age, 0);
2      component less_13 = LessThan()(age, 13);
3      component less_18 = LessThan()(age, 18);
4      component less_30 = LessThan()(age, 30);
5      component less_60 = LessThan()(age, 60);

```

- Se generan señales intermedias para que las condiciones sean mutuamente excluyentes:

```

1      signal is_less_0 = less_0.out;
2      signal is_less_13 = (1 - less_0.out) * less_13.out;
3      signal is_less_18 = (1 - less_0.out) * (1 - less_13.out) *
4          less_18.out;
5      signal is_less_30 = (1 - less_0.out) * (1 - less_13.out) * (1 -
6          less_18.out) * less_30.out;
7      signal is_less_60 = (1 - less_0.out) * (1 - less_13.out) * (1 -
8          less_18.out) * (1 - less_30.out) * less_60.out;
9      signal not_less_60 = 1 - less_60.out;

```

- Combinación ponderada de los mensajes:

```

1      mensaje <= is_less_0 * m1 +
2          is_less_13 * m2 +
3          is_less_18 * m3 +
4          is_less_30 * m4 +
5          is_less_60 * m5 +
6          not_less_60 * m6;

```

Como podemos observar, siempre se repite el mismo patrón en el que cada condición utiliza la negación de las condiciones anteriores y el resultado de la comparación actual para el caso de los *else/if*

En cuanto al *else*, como se explico en el caso simple, se observa que siempre es la negación de la suma de todas las condiciones anteriores, quedando reflejado con más claridad en este ejemplo

Para el caso del *switch* sería similar, ya que es una estructura semejante a la de *if/else if/else*.

Observaciones: En este ejemplo concreto se uso el operador lógico <, pero para cualquier otra operación lógica bastaría con aplicar la transformación especificada en el apartado de operadores lógicos y de comparación.

Condicionales anidados

```
1
2      // Código en C
3
4      if (age < 20) {
5          if (age < 10){
6              printf("Perfil: ninyo.\n");
7          }
8          else{
9              printf("Perfil: adolescente.\n");
10         }
11     } else if (age < 40){
12         if (age < 30){
13             printf("Perfil: adulto joven.\n");
14         }
15         else {
16             printf("Perfil: adulto mayor.\n");
17         }
18     }
19     else{
20         printf("Perfil: adulto mayor.\n");
21     }
```

- Por cada condición de igualdad/desigualdad se crea un componente correspondiente, como en los casos anteriores.

```
1      component less_10 = LessThan()(age,10);
2      component less_20 = LessThan()(age,20);
3      component less_30 = LessThan()(age,30);
4      component less_40 = LessThan()(age,40);
```

- Partiendo de lo citado en los apartados anteriores, tenemos que generar las condiciones excluyentes.

```
1      signal is_less_20 = less_20.out;
2      signal is_less_10 = is_less_20 * less_10.out;
3      signal isnt_less_10 = is_less_20 * (1 - less_10.out);
4
5      signal is_less_40 = less_40.out;
6      signal is_less_30 = is_less_40 * less_30.out;
7      signal isnt_less_30 = is_less_40 * (1 - less_30.out);
8
9
10     signal is_greater_40 = (1-is_less_40);
```

- Combinación ponderada de los mensajes:

```
1      mensaje <== is_less_10 * m1 + isnt_less_10 * m2 +
2      is_less_30 * m3 + isnt_less_30 * m4 +
3      is_greater_40 * m5;
```

7. Operadores de desplazamiento

Los operadores de desplazamiento en C funcionan de la siguiente manera:

Imaginemos que tenemos el número 2 que queremos desplazar 2 bits.

- 1. Dado un número decimal 2, transformarlo a binario, que es 0010.
- 2. Con el n (número de bits a desplazar), desplazamos los dos últimos bits.
- 1000 (8 en decimal) es el número que se obtiene desplazando los bits.

*Observación: Podemos observar que para obtener el número equivalente del desplazamiento de bits en decimal se consigue con $x * 2^n$ en el caso de desplazamiento a la izquierda y con $x/2^n$ en el caso de desplazamiento a la derecha. Sin embargo, estas operaciones son muy costosas, por lo que optaremos por otra opción.*

Para conseguir una equivalencia entre estos operadores, directamente transformamos el número decimal en binario y, en función de si se trata de un desplazamiento a la izquierda o a la derecha, despreciamos y añadimos por un lado o por otro.

Función Bits2Num()

```

1      template Bits2Num(N){
2          signal input  binario[N];
3          signal output decimal;
4
5          signal potencia[N];
6          signal sum[N];
7
8          // Primera potencia = 1
9          potencia[0] <== 1;
10         sum[0] <== binario[0] * potencia[0];
11
12         for (var i = 1; i < N; i++) {
13             potencia[i] <== potencia[i-1] * 2;           // potencia
14             sum[i] <== sum[i-1] + binario[i] * potencia[i]; // acumulacion
15         }
16
17         decimal <== sum[N-1];
18     }

```

Desplazamiento a la izquierda

Explicaremos un ejemplo de la aplicación que haremos para $11 \ll 2$.

- Dado un número decimal, lo convertimos a binario, en este caso 11:

00001011

- Con el número en binario, desplazamos 2 bits a la izquierda:

00001011 → 00101100

- Como podemos ver, los últimos dos bits se han transformado en 0, por lo que la idea es añadir un número n de ceros al final del array que representa el número binario.

```

1      // Código Circom
2      template desplazamiento_izq(N) {
3          signal input numero_decimal;
4          signal input desplazamiento;
5          signal output salida;
6
7
8          assert(N >= desplazamiento);

```

```

9
10     component n2b = Num2Bits(N);
11     n2b.in <== numero_decimal;
12
13     signal binario[N];
14
15     // Desplazar bits a la izquierda
16     for (var i = 0; i < N - desplazamiento; i++) {
17         binario[i] <== n2b.out[i+desplazamiento];
18     }
19
20     // Rellenar con ceros al inicio
21     for (var i = N - desplazamiento; i < N; i++) {
22         binario[i] <== 0;
23     }
24
25     component b2n = Bits2Num(N);
26     b2n.in <== binario;
27
28     salida <== b2n.out;
29 }
30
31 component main = desplazamiento_izq(N);

```

Desplazamiento a la derecha

Explicaremos un ejemplo de la aplicación que haremos para $11 \gg 2$

- Dado un número decimal, lo convertimos a binario, en este caso 11:

00001011

- Con el número en binario, desplazamos 2 bits a la derecha:

00001011 → 00000010

- Como podemos ver, los últimos dos bits se han despreciado, añadiendo dos ceros al principio del número binario.

```

1 // Código Circom
2 template desplazamiento_dcha(N) {
3     signal input numero_decimal;
4     signal input desplazamiento;
5     signal output salida;
6
7     assert(N >= desplazamiento);
8
9     component n2b = Num2Bits(N);
10    n2b.in <== numero_decimal;
11
12    signal binario[N];
13
14    // Rellenar con ceros al final
15    for (var i = 0; i < desplazamiento; i++) {
16        binario[i] <== 0;
17    }
18
19    // Desplazar bits a la derecha
20    for (var i = 0; i < N-desplazamiento; i++) {
21        binario[i+desplazamiento] <== n2b.out[i];
22    }
23

```

```

24     component b2n = Bits2Num(N);
25     b2n.in <== binario;
26
27     salida <== b2n.out;
28 }
29
30 component main = desplazamiento_dcha(N);

```

Observación: Hay que asegurar de que el tamaño de N sea mayor o igual que el desplazamiento que se quiere realizar.

Operación	Deducción
$e = e_1 \ll e_2$	Para un component $izq = \text{desplazamiento_izq}(n)(a,b)$: $\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \ll e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} \leq izq.out\}, v_{aux})}$
$e = e_1 \gg e_2$	Para un component $izq = \text{desplazamiento_dcha}(n)(a,b)$: $\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \gg e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} \leq dcha.out\}, v_{aux})}$